

《面向对象程序设计》之——

多 态

连 盛

shenglian@fzu.edu.cn

计算机与大数据学院

福州大学

本章要点

- 什么是多态性？
- 多态性的意义
- 多态性的实现方法
- 实现多态性的有关规定

多态性的概念

多态性是面向对象的程序设计的关键技术。

多态性：调用同一个函数名，可以根据需要但实现不同的功能。

多态性 { 编译时的多态性（函数重载）
运行时的多态性（虚函数）

运行时的多态性是指在程序执行之前，根据函数名和参数无法确定应该调用哪一个函数，必须在程序的执行过程中，根据具体的执行情况来动态地确定

多态性的概念

■ 多态性要解决的问题

- 静态多态性（重载，已解决）

- 动态多态性：虚函数

- 当一个基类被继承为不同的派生类时，各派生类可以使用与基类成员相同的成员名。如果在运行时用同一个成员名调用类对象成员，会调用哪一个对象的成员呢？

可以将一个派生类对象的地址赋给基类的指针变量。

基类指针

Base *basep;

基类对象

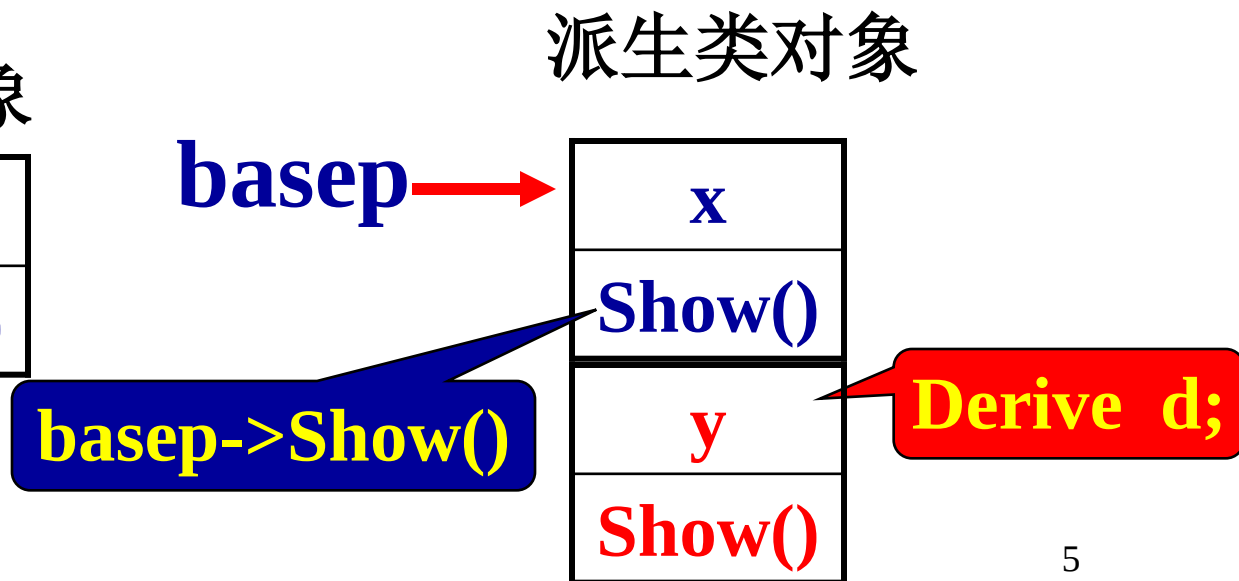
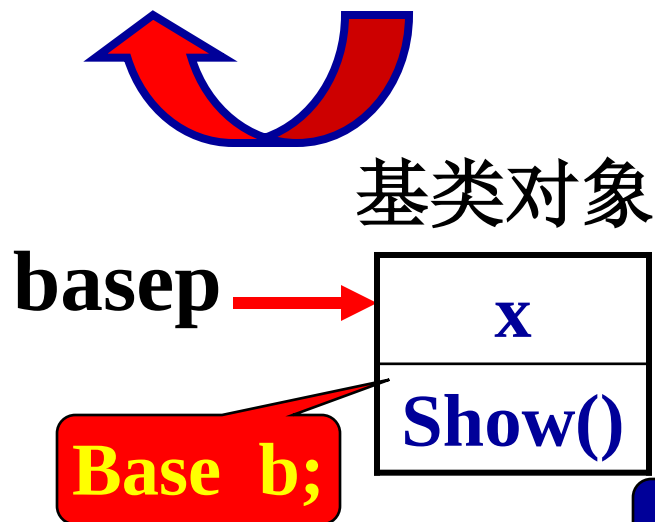
basep=&b;

派生类对象

basep = &d;

basep ->Show();

basep只能引用从基类继承来的成员。



```
class Point{
    float x,y;
public: Point(){
    Point(float i,float j){    x=i;    y=j;    }
    float area(void) {        return 0.0;    }
};

const float Pi=3.14159;
class Circle:public Point{    //类Point的派生类
    float radius;
public: Circle(float r){    radius=r;    }
    float area(void) {    return Pi*radius*radius;    }
};

void main(void)
{    Point *pp;    //基类指针，可以将派生类对象的地址赋给基类指针
    Circle c(5.4321);
    pp=&c;
    cout<<pp->area ()<<endl;    //调用的是基类中有的公有函数
}
```

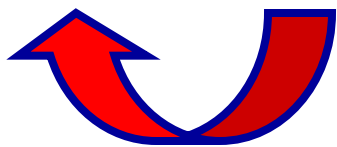
在基类和派生类中具有相同的公有函数area()。

在这种情况下，使用基类的指针时，只能访问从相应基类中继承来的成员，而不允许访问在派生类中增加的成员。输出为 0

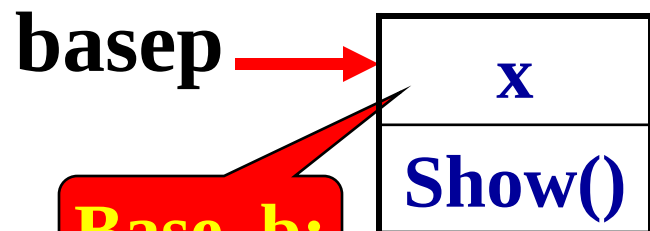
Base *basep;

basep=&b;

basep = &d;



基类对象



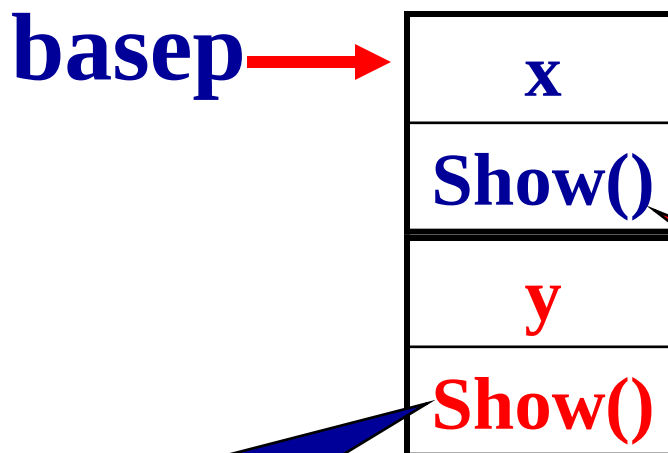
Base b;

basep ->Show();

即指向派生类新增的成员函数

需要将基类中的Show()
说明为虚函数

派生类对象



Derive d;

basep->Show()

若要访问派生类中相同名字的函数，必须将基类中的同名函数定义为虚函数，这样，将不同的派生类对象的地址赋给基类的指针变量后，就可以动态地根据这种赋值语句调用不同类中的函数。

```
class Point{ float x,y;
public: Point(){
    Point(float i,float j){    x=i;    y=j;    }
    virtual float area(void) { return 0.0; }
};
```

声明为虚函数

```
const float Pi=3.14159;
class Circle:public Point{           //类Point的派生类
    float radius;
public: Circle(float r){ radius=r;    }
    float area(void) { return Pi*radius*radius;}
};
```

虚函数再定义

```
void main(void)
{   Point *pp;           //基类指针，可以将派生类对象的地址赋给基类指针
    Circle c(5.4321);
    pp=&c;
    cout<<pp->area ()<<endl; //调用虚函数
}
```

调用虚函数

输出： 92.7011

将area()声明为虚函数，编译器对其进行动态关联，按照实际对象c调用了Circle中的函数area()。使Point类中的area()与Circle类中的area()有一个统一的接口。

虚函数的定义和使用

可以在程序运行时通过调用相同的函数名而实现不同功能的函数称为虚函数。定义格式为：

virtual <type> FuncName(<ArgList>);

一旦把基类的成员函数定义为虚函数，由基类所派生出来的所有派生类中，该函数均保持虚函数的特性。

在派生类中重新定义基类中的虚函数时，可以不用关键字**virtual**来修饰这个成员函数。

- 虚函数是用关键字**virtual**修饰的某基类中的**protected**或**public**成员函数。它可以在派生类中重新定义，以形成不同版本。
- 只有在程序的执行过程中，依据指针具体指向哪个类对象，或依据引用哪个类对象，才能确定激活哪一个版本，实现动态关联。

```
class A{
protected:      int x;
public:  A(){x=1000;}
        virtual void print(){      cout <<“x=”<<x<<‘\t’;   }//虚函数
};

class B:public A{      int y;
public:  B() { y=2000;}
        void print(){      cout <<“y=”<<y<<‘\t’;   }//派生虚函数
};

class C:public A{      int z;
public:  C(){z=3000;}
        void print(){      cout <<“z=”<<z<<‘\n’;   }//派生虚函数
};

void main(void )
{  A  a, *pa;
   B  b; C  c;
   a.print();  b.print(); c.print(); //静态调用
pa=&a;  pa->print();//调用类A的虚函数
pa=&b;  pa->print();//调用类B的虚函数
pa=&c;  pa->print();}//调用类C的虚函数
```

```
class Base {  
public :  
virtual int Set(int a, int b)  
{ ..... }  
....  
};
```

int Set(int ,int)是虚函数

```
class Derive:public Base{  
public :  
int Set(int x, int y)  
{ ..... }  
....  
};
```

```
class Base {  
public :  
virtual int Set(int a, int b)  
{ ..... }  
....  
};
```

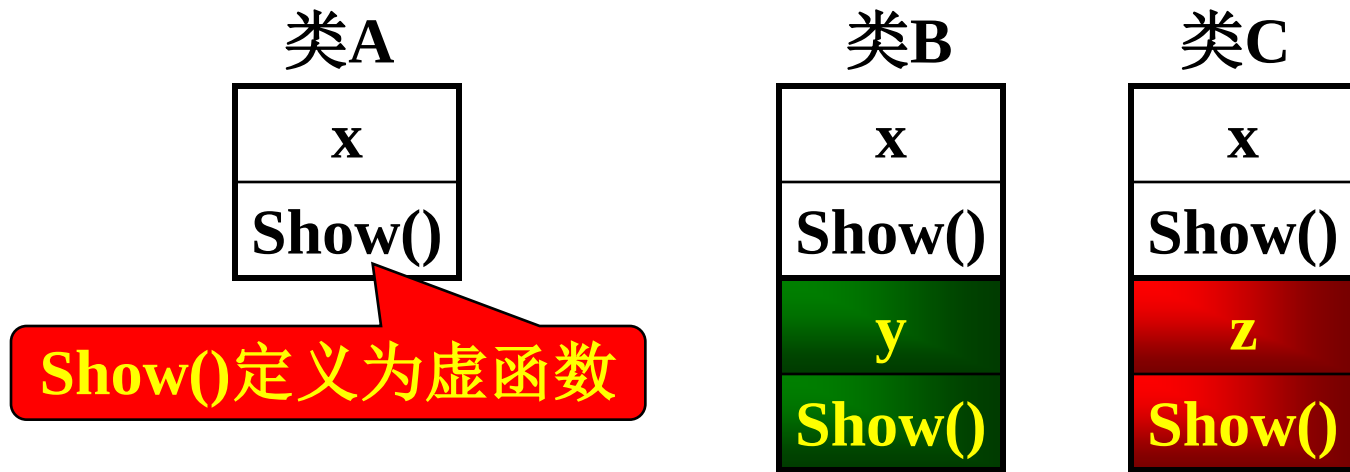
两个Set()函数参数
不一致，是重载，
不是虚函数

```
class Derive:public Base{  
public :  
int Set(int x, int y, int z)  
{ ..... }  
....  
};
```

关于虚函数，说明以下几点：

1、当在基类中把成员函数定义为虚函数后，在其派生类中定义的虚函数必须与基类中的虚函数同名，参数的类型、顺序、参数的个数必须一一对应，函数的返回的类型也相同。若函数名相同，但参数的个数不同或者参数的类型不同时，则属于函数的重载，而不是虚函数。若函数名不同，显然这是不同的成员函数。

2、实现这种动态的多态性时，必须使用**基类类型**的**指针变量**，并使该指针**指向不同的派生类对象**，并通过调用指针所指向的虚函数才能实现动态的多态性。



类B与类C均为类A的公有派生。

```

A  *p;
B   b;
C   c;

p=&b;
p->Show();

p=&c;
p->Show();
    
```

即在程序运行时，通过赋值语句实现多态性

3、虚函数必须是类的一个非静态成员函数，不能是静态的成员函数。也不能是友元函数。

4、在派生类中**没有重新定义虚函数**时，与一般的成员函数一样，当调用这种派生类对象的虚函数时，**则调用其基类中的虚函数**。

虚函数的一些性质

- 为什么虚函数必须是类的成员函数？
 - 虚函数诞生的目的就是为了实现多态，在类外定义虚函数毫无实际用处。
- 为什么类的静态成员函数不能为虚函数？
 - 考虑静态成员函数定义：内存只有一份拷贝
- 可以将析构函数定义为虚函数。为什么？
 - 是一个优秀的习惯，从`delete`的角度思考。

可把析构函数定义为虚函数，但是，不能将构造函数定义为虚函数。

析构函数可以是虚函数

当基类指针指向派生类对象，并且通过该指针删除对象时，如果基类的析构函数是虚函数，那么将调用派生类的析构函数，进而确保派生类部分也被正确清理。

构造函数不能是虚函数

虚函数的实现依赖于虚函数表和一个隐藏在对象内部的指针。这个表和指针在运行时初始化，即对象已经创建之后。而构造函数负责创建对象，此时对象还未完全形成，该表和指针还未初始化，因此构造函数无法作为虚函数。

6、虚函数与一般的成员函数相比较，**调用时的执行速度要慢一些**。为了实现多态性，在每一个派生类中均要保存相应虚函数的入口地址表，函数的调用机制也是间接实现的。因此，除了要编写一些通用的程序，并一定要使用虚函数才能完成其功能要求外，通常不必使用虚函数。

7、一个函数如果被定义成虚函数，则不管经历多少次派生，仍将保持其虚特性，以实现“一个接口，多个形态”。

虚函数的访问

用基指针访问与用对象名访问

用基指针访问虚函数时，指向其实际派生类对象重新定义的函数。实现动态关联。

通过一个对象名访问时，只能静态关联。即由编译器在编译的时候决定调用哪个函数。

虚析构造函数 (1/3)

- 构造函数的执行顺序：从上到下
- 析构函数的执行顺序：从下到上
- 创建对象时要执行正确的构造函数
- 撤销对象时要执行正确的析构函数
- 问题：动态对象的创建和撤销

虚析构函数 (2/3)

- 动态对象的创建

`new ClassName(...);`

`Car** cars = new Car*[n];`

- `ClassName` 指明了要调用的构造函数

- 动态对象的撤销

`delete 基类指针;`

- 如果基类指针指向的是派生类的对象呢？

虚析构函数 (3/3)

- 解决方案：把析构函数定义为虚函数
对于

`delete 基类指针;`

`//该类的析构函数为虚函数`

- 程序会根据基类指针指向的对象的类型确定要调用的析构函数
- 基类的析构函数为虚函数，所有派生类的析构函数都是虚函数

例子3: 虚析构函数的例子 (1/7)

```
//文件employee.h, 定义基类Employee
#include <iostream>
#if !defined __EMPLOYEE__H__
#define __EMPLOYEE__H__
class Employee
{public:
    Employee()
    {cout<<"Employee begin!"<<endl;
    }
    virtual ~Employee()
    {cout<<"Employee end!"<<endl;
    }
};
#endif
```

例子3: 虚析构函数的例子 (2/7)

```
//文件programmer.h, 定义派生类Programmer
#include <iostream>
#include <string.h>
#include "employee.h"
class Programmer: public Employee
{public:
    Programmer(char *str)
    {cout<<"Programmer begin!"<<endl;
      name = new char[strlen(str)+1];
      strcpy(name, str);
    }
    ~Programmer()
    {delete [] name;
      cout<<"Programmer end!"<<endl;
    }
private:
    char *name;
};
```

例子3: 虚析构函数的例子 (3/7)

```
//文件accountant.h, 定义派生类Accountant
#include <iostream.h>
#include "employee.h"
class Accountant: public Employee
{public:
    Accountant(int n)
    {cout<<"Accountant begin!"<<endl;
      age = n;
    }
    ~Accountant()
    {cout<<"Accountant end!"<<endl;
    }
private:
    int age;
};
```

例子3: 虚析构函数的例子 (4/7)

```
//文件ex15_4.cpp
#include "accountant.h"
#include "programmer.h"
const int MAX = 100;
main()
{ int no;
  //声明储存雇员信息的数组
  Employee *ptr[MAX], *tptr;
  int ENum = 0;
  char name[100];
  int age;
  for (int i=0; i<MAX; i++)
    ptr[i] = NULL;
```

例子3: 虚析构函数的例子 (5/7)

```
//输入雇员信息
cout<<"Input employees' info:"<<endl;
cout<<"1 --- Programmer"<<endl
    <<"2 --- Accountant"<<endl
    <<"0 --- exit"<<endl;
cin>>no;
while (no)
{switch (no)
{case 1:
    cout<<"Please input name:";
    cin>>name;
    //创建程序员对象
    tptr = new Programmer(name);
    ptr[ENum++] = tptr;
    break;
```

例子3: 虚析构函数的例子 (6/7)

case 2:

```
cout<<"Please input his or her age:";
```

```
cin>>age;
```

```
//创建会计对象
```

```
tptr = new Accountant(age);
```

```
ptr[ENum++] = tptr;
```

```
break;
```

```
default:
```

```
break;
```

```
}
```

```
cout<<"Input another employee's info:"
```

```
<<endl;
```

```
cout<<"1 --- Programmer"<<endl
```

```
<<"2 ---Accountant"<<endl
```

```
<<"0 --- exit"<<endl;
```

```
cin>>no; }
```

例子3: 虚析构函数的例子 (7/7)

```
//撤销所有雇员对象  
for (i=0; i<ENum; i++)  
{  
    delete ptr[i];  
}  
return 0;  
}
```

```

class Point{ float x,y;
public: Point(){
    Point(float i,float j){    x=i;    y=j;    }
    virtual float area(void) { return 0.0; }//声明为虚函数
};

const float Pi=3.14159;
class Circle:public Point{    //类Point的派生类
    float radius;
public: Circle(float r){ radius=r;    }
    float area(void) { return Pi*radius*radius;}//虚函数再定义
};

void main(void)
{   Point *pp;    //基类指针，可以将派生类对象的地址赋给基类指针
    Circle c(5.4321);
    cout<<c.area()<<endl;
    cout<<c.Point::area()<<endl;
    cout<<c.Circle::area ()<<endl;
}

```

输出: 92.7011
0
92.7011

用对象名调用area()

可见，利用对象名进行调用与一般非虚函数没有区别。

纯虚函数

在基类中不对虚函数给出有意义的实现，它只是在派生类中有具体的意义。这时基类中的虚函数只是一个入口，具体的目的地由不同的派生类中的对象决定。这个虚函数称为**纯虚函数**。

```
class <基类名>
{
    virtual <类型><函数名>(<参数表>)=0;
    .....
};
```

virtual void print()=0; //定义纯虚函数

抽象类

```
class A{
protected:    int x;
public:  A(){x =1000;}
        virtual void print()=0; //定义纯虚函数
};
```

```
class B:public A{ //派生类
```

```
private:  int y;
```

```
public:  B(){ y=2000;}
```

```
        void print(){cout <<"y="<<y<<"\n";} //重新定义纯虚函数
```

```
};
```

```
class C:public A{ //派生类
```

```
        int z;
```

```
public:  C(){z=3000;}
```

```
        void print(){cout <<"z="<<z<<"\n";} //重新定义纯虚函数
```

```
};
```

```
void main(void )
```

```
{  A *pa;      B b;   C c;
```

```
  pa=&b;  pa->print(); pa=&c;  pa->print();
```

```
  A a;  pa=&a;  pa->print();
```

```
}
```

不能定义抽象类的对象

y=2000

z=3000

- 1、在定义纯虚函数时，不能定义虚函数的实现部分。
- 2、把函数名赋于0，本质上是将指向函数体的指针值赋为初值0。与定义空函数不一样，空函数的函数体为空，即调用该函数时，不执行任何动作。在没有重新定义这种纯虚函数之前，是不能调用这种函数的。

3、把至少包含一个纯虚函数的类，称为抽象类。这种类只能作为派生类的基类，不能用来说明这种类的对象。

其理由是明显的：因为虚函数没有实现部分，所以不能产生对象。但可以定义指向抽象类的指针，即指向这种基类的指针。当用这种基类指针指向其派生类的对象时，**必须在派生类中重载纯虚函数**，否则会产生程序的运行错误。

4、在以抽象类作为基类的派生类中必须有纯虚函数的实现部分，即必须有重载纯虚函数的函数体。否则，这样的派生类也是不能产生对象的。

综上所述，可把纯虚函数归结为：抽象类的唯一用途是为派生类提供基类，纯虚函数的作用是作为派生类中的成员函数的基础，并实现动态多态性。

回顾

- 静态多态性与动态多态性
- 动态多态性与虚函数
- 虚析构函数
- 纯虚函数与抽象类

抽象基类和纯虚函数 (1/4)

- 实现多态性的前提
 - 需要有共同的基类
 - 需要在基类中定义共同的接口
 - 接口要定义为虚函数
- 如果基类的接口没办法实现怎么办?
 - 如形状类shape的成员函数area

抽象基类和纯虚函数 (2/4)

■ 解决方法

- 不实现这些接口：纯虚函数
- 包含纯虚函数的类：抽象基类

■ 抽象基类不能实例化对象

- 形状类 `shape` 的对象是什么？

抽象基类和纯虚函数 (3/4)

■ 纯虚函数

- 没有函数体的“初始化为0”的函数

//纯虚函数

```
virtual double area() const = 0;
```

- 纯虚函数不需要实现，不能被执行
- 通过定义、重定义或继承获得

抽象基类和纯虚函数 (4/4)

- 抽象类和具体类
 - 包含纯虚函数（且未对其重定义）的类不能实例化对象，是抽象类
 - 如果一个类不是抽象类，则是具体类。
- 抽象类存在的意义是作为其它类的基类，也叫抽象基类

例子2: 抽象基类的例子 (1/6)

```
//文件shape.h, 定义抽象基类Shape
#ifndef __SHAPE__H__
#define __SHAPE__H__
#include <iostream>
class Shape
{public:
    virtual double area() const = 0;
    virtual void show() const = 0;
};
#endif
```

例子2: 抽象基类的例子 (2/6)

```
//文件circle.h, 定义派生类Circle
#ifndef __CIRCLE_H__
#define __CIRCLE_H__
#include "shape.h"
#define PI 3.1416
class Circle :public Shape
{public:
    Circle(double = 0.0,
           double = 0.0, double = 1.0) ;
    double area() const;
    void show() const;
private:
    double x,y;
    double r;
};
#endif
```

例子2: 抽象基类的例子 (3/6)

```
//文件circle.cpp, 实现类Circle
#include "circle.h"
Circle::Circle(double a,
               double b, double c)
{
    x = a;
    y = b;
    r = c;
}
double Circle::area() const
{
    return PI*r*r;
}
void Circle::show() const
{
    cout<<"I am a Circle: ";
}
// Circle是具体类
```

例子2: 抽象基类的例子 (4/6)

```
//文件rectangle.h, 定义派生类Rectangle
#ifndef __RECTANGLE__
#define __RECTANGLE__
#include "shape.h"
class Rectangle :public Shape
{public:
    Rectangle(double = 1.0, double = 1.0);
    double area() const;
    void show() const;
private:
    double length;
    double width;
};
#endif
```

例子2: 抽象基类的例子 (5/6)

```
//文件rectangle.cpp, 实现类Rectangle
#include "rectangle.h"
Rectangle::Rectangle(double a, double b)
{ length = a;
  width = b;
}
double Rectangle::area() const
{
  return length*width;
}
void Rectangle::show() const
{
  cout<<" I am a Rectangle: ";
} // Rectangle是具体类
```

例子2: 抽象基类的例子 (6/6)

```
//文件ex15_3.cpp, 测试类Shape的虚函数
#include "circle.h"
#include "rectangle.h"
void callArea(Shape &obj)
{obj.show(); //动态绑定
  cout<<"area = "<<obj.area()<<endl;
} // obj.area() 动态绑定
main()
{Circle cir(0.0, 0.0, 2.5);
  Rectangle rec(2.4, 5.3);
  //能用Shape类声明对象吗?
  callArea(cir);
  callArea(rec);
  return 0;
}
```

程序运行结果

I am a Circle: area = 19.635

I am a Rectangle: area = 12.72

虚基类

多基派生中的多条路径具有公共基类时，在这条路径的汇合处就会因对公共基类产生多个拷贝而产生同名函数调用的二义性。

解决这个问题的办法就是把**公共基类定义为虚基类**，使由它派生的多条路径的汇聚处只产生一个拷贝。

```
class Base{ };  
  
class A : public Base{ };  
  
class B: public Base{ };  
  
class C: public A, public B{ };
```

类C中继承了两个类Base，
即有两个类Base的实现部分，
在调用时产生了二义性。



由虚基类派生出的对象初始化时，**直接调用**虚基类的构造函数。因此，若将一个类定义为虚基类，则一定有正确的构造函数可供所有派生类调用。

用虚基类进行多重派生时，**若虚基类没有缺省的构造函数**，则在每一个派生类的构造函数中**都必须有对虚基类构造函数的调用**（且首先调用）。

多态例题：小动物们吃什么？

题目描述

请编写一个关于动物的类 `Animal`，该类包含一个公共函数 `eat()`，输出“Animal is eating.”

接下来，请编写五个子类 `Cat`，`Dog`，`Lion`，`Panda`，`Rabbit`，分别继承自父类 `Animal`，并覆盖 `eat()`。

`Cat` 的 `eat()` 函数应该输出“Cat is eating fish.”；

`Dog` 的 `eat()` 函数应该输出“Dog is eating bone.”；

`Lion` 的 `eat()` 函数应该输出“Lion is eating meat.”；

`Panda` 的 `eat()` 函数应该输出“Panda is eating bamboo.”；

`Rabbit` 的 `eat()` 函数应该输出“Rabbit is eating carrot.”。

(注意，所有输出的结尾只有句点，没有换行)。

先输入一个数字 `N`，表示小动物的个数，`N` 不超过 5。

随后输入 `N` 个小动物的名字。

最后，请在主函数中创建一个 `Animal` 类的指针，让它：

(1) 先指向第一个小动物对象，比如 `Cat` 对象，调用 `Cat` 的 `eat()` 函数，并输出结果；

(2) 然后将这个指针改为指向第二个小动物对象，比如 `Dog` 对象，再次调用 `Dog` 的 `eat()` 函数，并输出结果。

(3) 将这个指针改为指向第三个小动物对象……直到把 `N` 个小动物吃什么都输出完毕。

多态例题：小动物们吃什么？

输入格式：

首先输入一个数字N，随后输入N个小动物的名字，间隔符为空格。

输出格式：

按顺序输出每个小动物吃什么，注意，所有输出的结尾只有句点，没有换行。

输入样例：

在这里给出一组输入。例如：

```
4 Cat Lion Dog Rabbit
```

输出样例：

在这里给出相应的输出。例如：

```
Cat is eating fish.Lion is eating meat.Dog is eating bone.Rabbit is eating carrot.
```

多态例题：小动物们吃什么？

```
#include <iostream>
#include "string.h"
using namespace std;

class Animal{
public:
    virtual void eat(){
        cout << "Animal is eating.";
    }
};

class Cat : public Animal{
public:
    void eat()    {
        cout << "Cat is eating fish.";
    }
};
```

多态例题：小动物们吃什么？

```
class Dog : public Animal
{
public:
    void eat()
    {
        cout << "Dog is eating bone.";
    }
};

class Lion : public Animal
{
public:
    void eat()
    {
        cout << "Lion is eating meat.";
    }
};
```

```
int main(){
    int N = 0, i;
    char animal[10][30];
    Animal *ptr = new Cat();

    cin >> N;
    for (i = 0; i < N; i++){
        cin >> animal[i];
    }

    for (i = 0; i < N; i++){
        if (strcmp(animal[i], "Cat") == 0){
            ptr = new Cat();
            ptr->eat();
        }
        ..... // 判断不同动物的匹配
    }

    delete ptr;
    return 0;
}
```

学习目的检测

- 理解静态绑定和动态绑定的概念
- 了解多态性的意义和作用
- 掌握定义虚函数和抽象基类的方法
- 掌握实现多态性的方法以及使用多态性的时机
- 了解虚析构函数的作用

Thank you !